

Konzept für einen Linux-basierten Leitstellen-Client

Ziele und Rahmenbedingungen

Ziel des Projekts ist die **Entwicklung eines leistungsfähigen Leitstellen-Clients auf Linux-Basis** als Proof-of-Concept. Dieser Client soll die Kernfunktionen einer *112-Einsatzleitstelle* im Jahr 2025 abdecken – **ohne** die kostspielige proprietäre Leitstellen-Software nutzen zu müssen. Stattdessen wird auf ein eigenes, schlankes Leitstellen-System („LST-System“) gesetzt, das vollständig mit Open-Source-Komponenten realisiert wird. Ein Schwerpunkt liegt auf *hoher Performance, Stabilität und Unabhängigkeit von externen Diensten*, sodass das System auch bei Ausfall der Internetverbindung autark weiterläuft.

Wichtige Rahmenbedingungen:

- **Linux-Plattform:** Der Client soll auf gängigen Linux-Distributionen lauffähig sein (z.B. Rocky Linux/RHEL, Debian/Ubuntu und perspektivisch SUSE). Die bestehende Server-Infrastruktur basiert auf Rocky Linux VMs (unter VMware) und soll weiterverwendet werden.
- **Ansible-basierte Bereitstellung:** Alle Komponenten (Client, Datenbank, Schnittstellen) werden als **Ansible-Rollen** definiert und automatisiert installiert. Eine Ansible-Umgebung für Rocky-VMs ist bereits vorhanden und wird erweitert, um das gesamte LST-System reproducible bereitzustellen.
- **Open-Source-First:** Wo immer möglich, kommen Open-Source-Lösungen zum Einsatz – beispielsweise im Kartenmodul (OpenStreetMap statt proprietärem ArcGIS) ¹. Dies reduziert Kosten, vermeidet Herstellerabhängigkeiten und ermöglicht Einsicht in den Code (wichtig für Sicherheit).
- **Offline-Fähigkeit:** Das System soll *hochverfügbar* und **unabhängig von externen Netzen** sein. Alle kritischen Daten werden lokal vorgehalten, sodass der Betrieb der Leitstelle gewährleistet bleibt, selbst wenn Internet-Dienste nicht erreichbar sind. Externe Informationen (z.B. Wetterdaten) werden über dedizierte Gateways eingespielt und zwischengespeichert.

Funktionale Anforderungen

Der Leitstellen-Client muss alle wesentlichen Funktionen einer modernen Einsatzleitstelle unterstützen. Insbesondere sind folgende **Hauptfunktionen** vorgesehen:

- **Einsatzverwaltung:** Anlegen neuer Einsätze (Notrufe), Bearbeiten und Aktualisieren von Einsatzdetails (Einsatzort, Einsatzstichwort, Priorität etc.), sowie Abschließen/Schließen von Einsätzen nach Beendigung. Einsätze sollen mit Zeitstempeln und verantwortlichen Disponenten protokolliert werden.
- **Ressourcen- und Statusmanagement:** Verwalten der *Einsatzmittel* (z.B. Rettungswagen, Feuerwehreinheiten, Polizei) inklusive Echtzeit-Statusanzeige. Der Disponent muss den aktuellen Status jedes Fahrzeugs/Mittels sehen (frei, alarmiert, ausgerückt, an Einsatzstelle, im Einsatz, verfügbar etc.) und Statusänderungen (Rückmeldungen) schnell verbuchen können.
- **Kartenanzeige und Standortverfolgung:** Darstellung der Einsatzorte und der GPS-Positionen der Einsatzmittel auf einer digitalen Karte. Als Kartenhintergrund wird **OpenStreetMap (OSM)**

verwendet. Eingesetzte Einheiten werden als Icons auf der Karte dargestellt und aktualisieren sich mit neuen Positionsdaten (GPS-Telemetrie). So erhält der Disponent eine Lageübersicht in Echtzeit. Die bisher genutzte ArcGIS-Kartenlösung soll dadurch abgelöst werden.

- **Einsatzdokumentation und -kommunikation:** Möglichkeit, zu einem Einsatz Kommentare oder Meldungen zu hinterlegen (Einsatzprotokoll) und relevante Informationen (z.B. Notruftexte, Anruferinformationen) aufzunehmen. Ggf. modulare Erweiterbarkeit für Chat/Message-Funktionen an die Einsatzkräfte (Textnachrichten) oder Alarmierungsfunktionen.
- **Externe Informationsintegration:** Einbindung ergänzender Informationen via API, z.B. **Wetterdaten** für die Einsatzregion (Temperatur, Unwetterwarnungen) zur Lageeinschätzung. Solche Daten werden im Client angezeigt, aber **nicht** direkt vom Client aus dem Internet abgefragt (siehe Architekturbeschreibung unten).
- **Benutzerverwaltung und Sicherheit:** Anmeldung der Disponenten am System, Rechteverwaltung (z.B. nur Berechtigte dürfen Einsätze schließen oder Systemkonfiguration ändern). Sensible personenbezogene Daten (Anruferdaten etc.) müssen geschützt gespeichert und übertragen werden.
- **Erweiterbarkeit:** Die Client-Software soll **modular und erweiterbar** sein, um zukünftige Anforderungen aufzunehmen. Beispielsweise könnten später Module für **Anrufannahme** (Telefonie-Integration), **Alarmierungssysteme** oder **automatische Dispositionsvorschläge** (Vorschlag geeigneter Einsatzmittel basierend auf Standort und Verfügbarkeit) hinzugefügt werden. Die Architektur soll dies von Beginn an berücksichtigen.

Hinweis: Es werden **keine wesentlichen Funktionen ausdrücklich ausgeschlossen**. Das System soll langfristig alle klassischen Aufgaben einer Leitstelle erfüllen können. Für den **Proof-of-Concept** liegt der Fokus jedoch auf den oben genannten Kernfunktionen; Spezialfunktionen können in späteren Phasen ergänzt werden.

Systemarchitektur

Um die obigen Anforderungen zu erfüllen, wird eine **mehrschichtige Architektur** gewählt. Sie trennt *Client-Anwendung*, *Datenmanagement* und *externe Schnittstellen* in separate Komponenten, die klar definierte Aufgaben haben. Dies erhöht Sicherheit und erleichtert die Wartung. Die Hauptkomponenten der Architektur sind:

- **Zentrale Leitstellen-Datenbank (Server):** Ein relationaler Datenbankserver speichert **alle Leitstellendaten** persistent. Hier liegen die Stammdaten der Einsatzmittel, die Bewegungsdaten (Einsätze, Statushistorie), Positionsdaten sowie alle Informationen, die für den Betrieb nötig sind. Diese *eigene Leitstellen-Datenbank* hält idealerweise **sämtliche Daten vor, die nicht über externe Live-APIs abrufbar sind** (also alle kritischen Einsatz- und Ressourceninformationen). Die Datenbank fungiert als Single Source of Truth, auf die der Client zugreift. (Details zur Datenbank siehe nächster Abschnitt.)
- **Leitstellen-Client (Frontend):** Die Client-Anwendung läuft auf den Arbeitsplätzen der Disponenten (Linux-Desktops) und stellt die Benutzeroberfläche bereit. Der Client kommuniziert *ausschließlich* mit der internen Leitstellen-Datenbank – **nicht** direkt mit externen Diensten. Über diese DB-Schnittstelle kann der Client Einsätze anlegen/bearbeiten, Statusupdates schreiben oder lesen und Kartendaten abrufen. Da der Client keine direkten Internetzugriffe durchführt, bleibt er selbst bei Ausfall externer Netze funktionsfähig (er arbeitet dann mit den lokal vorliegenden Daten weiter). Auch sicherheitstechnisch reduziert dies die Angriffsfläche erheblich.
- **Gateway-Server (Schnittstellenmodul):** Für den Zugang zu **externen APIs** und Diensten wird ein separates Gateway bzw. Schnittstellen-Service betrieben. Dieses Modul läuft serverseitig (z.B. auf einer Rocky Linux VM) und übernimmt z.B. das periodische Abrufen von

Wetterinformationen oder anderen externen Datenquellen. Der Gateway-Server bereitet die erhaltenen Daten auf und schreibt sie in die Leitstellen-Datenbank (in definierte Tabellen).

Beispiel: Ein Wetter-Service ruft alle 10 Minuten die aktuellen Wetterdaten der Region von einem Wetter-API ab (z.B. OpenWeatherMap oder DWD) und aktualisiert eine Wetter-Tabelle in der DB. Der Client liest dann nur aus dieser Tabelle, anstatt selbst das Internet zu kontaktieren. Dieses Entkoppeln erhöht die Sicherheit und ermöglicht es, externe Daten zu puffern: Sollte die Internetverbindung gestört sein, stehen zumindest die *zuletzt bekannten* externen Informationen weiterhin zur Verfügung. Das Gateway-Modul kann für alle möglichen Schnittstellen genutzt werden (z.B. könnte man später auch Verkehrsdaten, Nachrichten oder Schnittstellen zu benachbarten Leitstellen hierüber einbinden) – es agiert als kontrollierte **Brücke nach außen**, während der eigentliche Leitstellenbetrieb intern weiterläuft.

- **Karten- bzw. Geodaten-Server (optional):** Da Kartenfunktionalität benötigt wird, ist zu entscheiden, wie die OSM-Karten bereitgestellt werden. Im einfachsten Fall können die Clients die Hintergrundkarten von öffentlichen OSM-Tile-Servern laden. Jedoch würde das Internetzugriff erfordern. Für *maximale Autarkie* und Performance bietet sich an, einen eigenen **OSM-Tile-Server** im Netzwerk zu betreiben. Dieser Server hält die benötigten Kartendaten (z.B. des Bundeslandes oder der gesamten Region) in einer lokalen PostGIS-Datenbank und liefert Kachelbilder an die Clients aus. Solch ein Tile-Server besteht aus einer OSM-Datenbank (PostgreSQL/PostGIS) und einem Rendering-Dienst (z.B. Mapnik + mod_tile/Apache) ², der auf Anfrage Kartenausschnitte als Bildkacheln generiert. *Alternativ* kann man auch einen einfacheren Ansatz wählen, etwa vorab gerenderte Karten (Rasterbild-Kacheln) der Region auf dem Server ablegen und per Webserver ausliefern, oder eine vektorbasierte Karte verwenden. Für das Proof-of-Concept könnte zunächst ein externer OSM-Server genutzt werden (unter Beachtung der Nutzungsbedingungen), aber perspektivisch ist ein **lokaler Kartenserver** für vollständige Offline-Fähigkeit sinnvoll.

Diese Komponenten kommunizieren folgendermaßen: **Der Leitstellen-Client verbindet sich über das interne Netzwerk zur Datenbank** (z.B. via PostgreSQL-TCP-Verbindung). Er sendet Lese- und Schreibenfragen (SQL-Queries) an die DB, um Einsätze und Status zu verwalten. Die **Gateway-Services** sind serverseitige Prozesse, die bei Bedarf externe HTTP-Anfragen durchführen, und das Ergebnis in die DB schreiben. Dadurch fließen alle Daten in die zentrale DB zusammen. Der Client kann z.B. die Wetterdaten einfach aus der DB abfragen und anzeigen.

Durch diese Architektur ergeben sich mehrere Vorteile: - **Sicherheit:** Der Client selbst hat keine direkte Internetverbindung – eventuelle Schwachstellen in externen APIs können die Client-App nicht kompromittieren. Das System ist intern abgeschottet; nur das Gateway agiert nach außen und lässt sich entsprechend streng kontrollieren (z.B. Firewall-Regeln erlauben nur dem Gateway ausgehend HTTPS). - **Performance:** Der lokale Datenbankserver ermöglicht schnelle Zugriffe im LAN ohne Latenz der WAN/Internet. Alle häufig genutzten Daten sind auf schnellen lokalen Datenträgern abgelegt. Zudem kann die Last verteilt werden: rechenintensive Rendering-Aufgaben (Kartengenerierung) oder API-Abfragen laufen nicht auf den Clients, sondern auf Servern. - **Unabhängigkeit:** Wenn „draußen die Welt untergeht“ – sprich die Internetverbindung ausfällt oder externe Dienste Probleme haben – bleiben **Client und Datenbank voll einsatzfähig**. Bereits bestehende Einsätze und Ressourceninformationen sind ja in der eigenen Datenbank vorhanden. Neue Einsätze können aufgenommen werden, Einheiten koordiniert werden, Karten ggf. aus dem Cache angezeigt werden. Nur frische externe Infos (Wetter, neue Online-Karten) würden in dem Fall temporär fehlen, was jedoch die Kernfunktion nicht lahmlegt. - **Skalierbarkeit und Erweiterung:** Die klare Trennung erlaubt es, weitere Clients hinzuzufügen (mehrere Disponenten-Arbeitsplätze greifen parallel auf dieselbe DB zu), sowie zusätzliche Gateway-Module zu integrieren, ohne die Clientsoftware ändern zu müssen. Auch der Austausch einer Komponente (z.B. anderer Kartendienst) ist möglich, solange die Schnittstelle (DB-Schema bzw. API) konsistent bleibt.

Technologieentscheidung für den Client

Ein zentrales Architektur-Element ist die Entwicklung der Client-Anwendung. Hier stellt sich die Frage: **In welcher Sprache und mit welchem Framework** soll der Leitstellen-Client implementiert werden, um auf Linux performant zu laufen und zukunftssicher zu sein?

Für den **Proof-of-Concept und die produktive Umsetzung** empfehlen wir einen **nativen Desktop-Client** auf Basis von **C++ und Qt** (Qt Framework). Diese Kombination bietet mehrere Vorteile:

- **Plattformunabhängiges GUI-Framework:** Qt ist ein bewährtes Toolkit, um Anwendungen für verschiedene Linux-Distributionen (sowie Windows/macOS) mit einer konsistenten Benutzeroberfläche zu entwickeln. Ein in C++/Qt geschriebener Client lässt sich relativ einfach auf RHEL-, Debian- und SUSE-Systemen deployen (Qt abstrahiert die Unterschiede der Windowing-Systeme). So können wir eine Codebasis pflegen und trotzdem unterschiedliche Linux-Umgebungen bedienen.
- **Hohe Performance und nativer Look&Feel:** C++ ist kompiliert und bietet maximale Ausführungsgeschwindigkeit – wichtig, da der Leitstellen-Client auch unter hoher Last (viele Einsätze, schnelle Updates) *flüssig* reagieren muss. Qt nutzt die native Grafik-API des OS (z.B. Qt Widgets oder Qt Quick mit OpenGL-Beschleunigung), was eine reaktionsschnelle Oberfläche ermöglicht. Insbesondere bei der Kartendarstellung und -interaktion zahlt sich native Performance aus.
- **Integrierte Funktionen für Kartendarstellung:** Qt bringt mit dem Modul **Qt Location** bereits Unterstützung für Karten und Geodaten mit. Über das Qt Location API kann direkt ein OpenStreetMap-gestützter Map Viewer in die Anwendung eingebettet werden. Qt bietet einen fertigen OSM-Plugin, der es der Anwendung erlaubt, OSM-Kartendienste anzusprechen ³. Dadurch lassen sich Kartenlayer, Marker (für Einsatzorte/Einsatzmittel) und sogar Routenberechnungen vergleichsweise leicht implementieren, ohne von Grund auf eine Kartenengine schreiben zu müssen.
- **Gute Erweiterbarkeit und Modularität:** Qt unterstützt ein Model-View-Controller (MVC/MVVM) Paradigma, was die Trennung von Geschäftslogik und GUI erleichtert. Wir können den Client so strukturieren, dass z.B. das Einsatzlisten-Modul, die Kartenansicht, das Status-Panel etc. getrennte Komponenten oder Widgets sind. Spätere Erweiterungen (neue Dialoge, Module) können als neue Widgets/Views hinzugefügt werden. Zudem verfügt Qt über ein Plugin-System, womit sich zur Laufzeit weitere Module laden ließen, falls benötigt.
- **Community und Support:** Qt ist im industriellen Umfeld erprobt und es gibt viel Dokumentation sowie Bibliotheken. Beispielsweise existieren Qt-Bindings für Datenbanken (Qt SQL Modul) und Netzwerke, was uns hilft, die PostgreSQL-DB anzubinden und eventuelle Netzwerkkommunikation zu handhaben.

Alternative Überlegung – Web-basierter Client: Als Alternative könnte man auch einen **Webclient** entwickeln, also eine Browser-basierte Anwendung (mittels HTML5/JavaScript, z.B. React oder Angular für das Frontend). Dieser würde über einen Webserver im Intranet ausgeliefert und im Browser der Linux-Clients laufen. Moderne Leitstellenprojekte wie das französische NexSIS-System setzen auf eine webbasierte Oberfläche ¹. Der Vorteil wäre, dass der Client sofort auf jeder Plattform mit Browser läuft und Updates zentral ausgerollt werden (einfach neue Version auf Server deployen, Clients laden die Seite neu). Auch lässt sich mit Frameworks wie Electron oder Tauri eine Web-App in einen Desktop-Container packen, um sie als "Programm" starten zu können. Für den gegebenen Rahmen bevorzugen wir jedoch die Qt/C++-Variante, da hier *Performance* und **Offline-Fähigkeit** im Vordergrund stehen. Ein Webclient wäre auf einen Browser und ggf. auf einen lokalen Webserver angewiesen; zudem ist die Hardwarebeschleunigung für Karten im Browser etwas schwerfälliger als in einer nativen Qt-

Anwendung. Für den PoC kann ein nativer Client zuverlässiger die Machbarkeit demonstrieren, dass ein performantes Linux-System für die Leitstelle möglich ist.

Implementierungsdetails des Clients: Unabhängig von der genauen UI-Technologie sollte der Client bestimmte Entwurfsmuster und Techniken nutzen:

- Verwendung eines **Datenbank-Abstraktionslayers** oder ORMs, um die Kommunikation mit der PostgreSQL-Datenbank zu kapseln. In Qt könnte z.B. `QSqlDatabase` mit passenden Treibern (QPSQL für PostgreSQL) eingesetzt werden, um SQL-Queries auszuführen. CRUD-Operationen (Erstellen/Lesen/Ändern/Löschen von Einsätzen etc.) werden so implementiert, dass sie effizient sind – z.B. vorbereitete Statements nutzen, um Performance zu steigern und SQL-Injection zu vermeiden.
- **Multithreading für Responsivität:** Die UI-Thread des Clients sollte nicht durch DB-Zugriffe oder lange Operationen blockiert werden. Daher werden zeitintensive Aufgaben (Datenbankabfragen, Rendern vieler Kartenelemente) in Hintergrund-Threads ausgelagert. Qt unterstützt dies (z.B. via `QtConcurrent` oder Worker-Threads). So bleibt die Oberfläche flüssig, auch wenn im Hintergrund z.B. eine große Einsatzliste geladen oder die Karte aktualisiert wird.
- **Push vs. Pull:** Um Daten aktuell zu halten, wäre es optimal, Änderungen **proaktiv** an den Client zu senden. Eine Möglichkeit ist die Nutzung der PostgreSQL-Notify/Listen-Funktionalität: Der Client kann sich per SQL `LISTEN` auf bestimmte Kanäle registrieren, und die DB (oder das Gateway) können via `NOTIFY` Nachrichten schicken, wenn z.B. ein neuer Einsatz angelegt wurde oder ein Fahrzeugstatus sich geändert hat. Die Client-App würde dieses Event empfangen und gezielt die betreffenden Daten neu laden. Dies ist effizienter als ständiges Polling aller Daten. Alternativ kann man ein Timer-basiertes Refresh implementieren (z.B. alle paar Sekunden bestimmte Tabellen abfragen), was einfacher aber lastintensiver ist. Für den PoC könnte Polling genügen; perspektivisch ist aber ein Event-getriebenes Update wünschenswert.
- **GUI-Design und Usability:** Die Oberflächengestaltung sollte sich an bewährten Leitstellen-UIs orientieren: Klare Aufteilung (z.B. Liste der laufenden Einsätze, Detailbereich zum ausgewählten Einsatz, Kartenfenster, Statusmonitor der Fahrzeuge). Farbgebung und Icons können zur schnellen Erfassung von Prioritäten oder Fahrzeugstatus dienen (z.B. Status-Farben Grün/Gelb/Rot). Da in Stresssituationen gearbeitet wird, ist eine intuitive Bedienung (Tastaturkürzel, kontextsensitive Menüs, visuelle Alarmierung bei neuen Notrufen) wichtig. Diese Aspekte fließen in die Entwicklung ein, auch wenn sie im PoC vllt. nur rudimentär umgesetzt werden.

Deployment des Clients: Um den Client auf verschiedenen Linux-Systemen lauffähig bereitzustellen, gibt es mehrere Ansätze. Mit Ansible können wir plattformspezifische Pakete installieren. Für Qt/C++-Anwendungen empfiehlt es sich, **statische Builds** oder **Container/Bundle**-Lösungen zu verwenden, um Abhängigkeiten zu minimieren. Eine Möglichkeit ist, den Client als **AppImage** bereitzustellen – ein AppImage läuft distributionsunabhängig, da alle Bibliotheken eingebettet sind. Alternativ kann man für RHEL-basierte Systeme ein RPM-Paket und für Debian/Ubuntu ein DEB-Paket erstellen. Ansible kann je nach Zielhost das passende Paket ausliefern. Wichtig: Die OpenGL/Graphik-Treiber müssen kompatibel sein; in heterogenen Umgebungen testet man am besten auf allen Zielsystemen. Für SUSE (später) könnte analog ein eigenes Paket gebaut oder ebenfalls AppImage verwendet werden. Da die Anwendung Open-Source-Komponenten nutzt, treten i.d.R. keine Lizenzkonflikte bei der Verteilung auf.

Datenbank-Design und Performance

Die **Leitstellen-Datenbank** ist das Herzstück des Systems. Hier werden alle Daten zentral gehalten. Wir wählen **PostgreSQL** als Datenbanksystem, aus mehreren Gründen: PostgreSQL gilt als eines der *fortschrittlichsten* und zuverlässigsten Open-Source-DBMS und wird in unzähligen modernen Anwendungen produktiv eingesetzt ⁴. Wichtig für uns: Es bietet hohe Leistung, ACID-Transaktionen

(für Datenkonsistenz auch bei parallelen Zugriffen) und umfangreiche Erweiterungsmöglichkeiten (z.B. Geodaten). In Performance-Benchmarks schneidet PostgreSQL im Vergleich zu MySQL/MariaDB hervorragend ab – ein Test zeigt PostgreSQL als klar schnellstes der drei, mit teils 50%+ Geschwindigkeitsvorteil gegenüber MariaDB und mehreren hundert Prozent gegenüber MySQL ⁵. Diese Performance ist für schnelle *Echtzeit-Abfragen* im Leitstellenbetrieb entscheidend.

Schema-Entwurf: Die Datenbank wird ein **relationales Schema** verwenden, um die Leitstellendaten abzubilden. Geplante Haupttabellen und Inhalte:

- **Einsatz** (`einsatz`): Enthält pro Notruf/Einsatz einen Datensatz mit Schlüssel (Einsatz-ID), Zeitstempeln (Eingang, Alarmierung, Abschluss), Einsatzort (Adresse oder Koordinaten), Kategorie/Stichwort (z.B. Feuer, Unfall, etc.), Priorität, Beschreibungstext, Status (offen/in Bearbeitung/abgeschlossen) etc. Verknüpft mit zugeordneten Ressourcen.
- **Einsatzmittel** (`einsatzmittel`): Listet alle verfügbaren Fahrzeuge/Einheiten mit ihren Stammdaten – Kennung (Rufname), Typ (RTW, NEF, Löschzug...), Standort/Wache, ggf. Personalstärke. Wichtig ist ein Feld für den *aktuellen Status* (1-6 System, z.B. Status 2 = ausgerückt, etc.) und eine Referenz, falls aktuell einem Einsatz zugeordnet.
- **Einsatzmittel-Statushistorie** (`status_history` o.ä.): Jedes Statuswechsel-Ereignis eines Fahrzeuges wird hier protokolliert (Fahrzeug X, Zeit, neuer Status, optional Bezug auf Einsatz). Dies erlaubt Nachverfolgung und Auswertung.
- **Positionsdaten** (`position`): Tabelle, die fortlaufend GPS-Positionsmeldungen der Fahrzeuge speichert (Fahrzeug-ID, Zeit, Koordinate (Längengrad, Breitengrad)). Alternativ könnte man die aktuelle Position auch als Attribut bei **Einsatzmittel** führen (letzte bekannte Position). Für historische Routen wäre jedoch eine eigene Tabelle sinnvoll. Diese Tabellen kann man mit PostGIS-Unterstützung ausstatten, sodass Geodaten-Felder (Point-Geometrien) genutzt werden und räumliche Abfragen (Entfernung, Umkreissuche) effizient mit Index möglich sind.
- **Benutzer/Disponenten** (`benutzer`): Nutzerverwaltung mit Login, Passwort (gehasht), Rolle/Rechte. So kann authentifiziert werden, welcher Disponent welche Aktion durchgeführt hat.
- **Wetterdaten** (`wetter`): Speichert externe Wetterinformationen, z.B. für bestimmte Regionen oder den Leitstellenbereich. Hier würde der Gateway-Service regelmäßig einen Datensatz aktualisieren (Temperatur, Wetterlage, Warnungen). Der Client liest z.B. immer den aktuellsten Eintrag aus dieser Tabelle.
- **Stammdaten-Tabellen:** z.B. für Fahrzeugtypen, Einsatzstichwort-Katalog, Krankenhäuser (Zielkliniken) etc., die im Einsatzfall benötigt werden. Diese ändern sich selten und können beim Systemstart geladen werden.

Das **Datenmodell** sollte sorgfältig normalisiert werden, um Dateninkonsistenzen zu vermeiden. Zugleich müssen für kritische Abfragen **Indizes** gesetzt werden. Beispielsweise: Index auf `einsatz.status` (für schnelle Abfrage aller offenen Einsätze), Index auf `einsatzmittel.status` (wer ist frei/besetzt), räumlicher Index (GiST) auf Koordinaten, um z.B. nächste verfügbare Einheit zum Einsatzort zu finden.

Zudem lässt sich mit **Views** oder Materialized Views arbeiten, um häufig benötigte aggregierte Sichten performant bereitzustellen – etwa eine View „alle freien RTW innerhalb von 20km um Ort X“. Solche optimierten Sichten könnten das Dispositionspersonal unterstützen. Für den PoC reicht aber vermutlich direkte Query.

Leistung und Optimierung: PostgreSQL erfordert für optimale Leistung einige Tuning-Maßnahmen auf dem Server (z.B. anpassen von `shared_buffers`, Work Memory etc., basierend auf RAM). Ansible kann nach Installation eine optimierte `postgresql.conf` verteilen, die auf die VM-Größe zugeschnitten ist. Die Datenbank sollte auf einer **SSD** liegen (für schnelle I/O-Zugriffe). In Lasttests

könnte man typische Anfragen simulieren (z.B. 100 gleichzeitige Einsatz-Updates) und prüfen, ob die Reaktionszeiten im niedrigen Millisekundenbereich bleiben – was bei kleinem Datenbestand und gutem Index der Fall sein sollte.

Um die Performance weiter zu erhöhen, können wir *Caching* in Betracht ziehen: Da der Client ständig bestimmte Daten braucht (z.B. Liste der aktiven Einsätze, Fahrzeugliste), könnte der Client diese einmal laden und dann im Speicher aktualisieren statt jedes Mal komplett von der DB zu lesen. Außerdem bietet PostgreSQL Mechanismen wie Query Plans Caching und die bereits erwähnte `LISTEN/NOTIFY`-Funktion, die effizientere Updates ermöglicht. Falls notwendig, könnte auch ein In-Memory-Cache wie **Redis** eingebunden werden, um beispielsweise standortbezogene Berechnungen schneller zu machen – aber wahrscheinlich ist das Overkill für den geplanten Umfang, zumindest anfänglich.

Geodaten und PostGIS: Sobald Kartendarstellung und Standortberechnungen ins Spiel kommen, bietet die Erweiterung **PostGIS** für PostgreSQL enorme Vorteile. PostGIS erweitert PostgreSQL um Geodatentypen (Punkt, Polygon etc.) und Geo-Funktionen. Beispielsweise kann man Abfragen formulieren wie „SELECT * FROM einsatzmittel WHERE ST_DWithin(position, <Einsatzort>, 5000) AND status = 'frei'“ um alle freien Fahrzeuge im 5km-Radius zu finden. Diese rechenintensiven Operationen werden durch PostGIS intern optimiert, insbesondere wenn spatial Indizes vorhanden sind. Für die OSM-Integration (Tile-Server) ist PostGIS ohnehin erforderlich, da der Kartenrenderer die OSM-Daten aus einer PostGIS-DB zieht ². Im Kontext unserer Leitstelle würde PostGIS also doppelt Nutzen bringen: einmal für interne Abfragen (z.B. Routing oder nächstes Fahrzeug berechnen) und einmal für den Kartenserver (sofern wir einen eigenen betreiben).

Datenhaltung und Ausfallsicherheit: Da die Leitstellen-DB mission-critical ist, sind Backups und ggf. Hochverfügbarkeit zu bedenken. In einer späteren Phase kann man z.B. **Streaming Replication** von PostgreSQL einrichten, um einen sekundären DB-Server als Hot-Standby zu haben. So könnte bei Ausfall des Primärservers nahtlos umgeschaltet werden. Für den PoC reicht regelmäßiges Backup (z.B. per Ansible cron job `pg_dump` täglich). Wichtig ist, dass wenn die Netzwerkanbindung zum DB-Server mal kurz wegfällt, der Client das abfängt und z.B. eine Fehlermeldung zeigt oder in Read-Only-Modus geht, bis die Verbindung wieder da ist. In einem echten Setup wären DB und App vermutlich im gleichen lokalen Netz, sodass Verbindungsabbrüche selten sind.

Zusammenfassend bietet das gewählte Datenbanksystem eine robuste Grundlage: **PostgreSQL mit PostGIS** liefert die notwendige *Performance, Flexibilität und räumliche Datenauswertung*, vollständig Open-Source und ohne Lizenzkosten ⁶. Durch gutes Schema-Design und Optimierung werden schnelle Datenbankabfragen sichergestellt, was ein Kernziel (Geschwindigkeit) des Projekts ist.

Integration von OpenStreetMap (Kartenmodul)

Für die Kartenanzeige setzen wir vollständig auf **OpenStreetMap (OSM)** als Hintergrundkarte. OSM bietet freie, aktuelle Geodaten unter offener Lizenz, was perfekt zum Open-Source-Ansatz passt. Die bisher verwendete proprietäre Esri-ArcGIS-Lösung wird damit abgelöst – das spart Kosten und vermeidet Abhängigkeit von einem einzelnen Anbieter. Das OSM-Ökosystem erlaubt zudem, die Karten **offline vorzuhalten**, was für die Ausfallsicherheit entscheidend ist.

Kartenhintergrund einbinden: In der Client-Anwendung wird ein Karten-Widget integriert. Mit Qt lässt sich beispielsweise ein `QML Map` Element nutzen, das via Qt Location die OSM-Karten laden kann. Über den OSM-Plugin-Schlüssel `"osm"` greift Qt Location auf OpenStreetMap-Dienste zu ³. Out-of-the-box kann man damit Kartenausschnitte laden, Marker setzen (für Einsatzorte, Fahrzeugpositionen) und die Karte bewegen/zoomen. Die OSM-Daten (Tiles) stammen standardmäßig von frei nutzbaren

Servern – in Qt sind z.B. Thunderforest oder direkt OSM per Default eingebunden. Wichtig ist, die **Nutzungsbedingungen** zu beachten: öffentliche OSM-Tile-Server haben Limits, daher im Produktivfall entweder einen eigenen Tile-Server nutzen oder einen externen Tile-Dienst mit API-Key.

Für den **Proof-of-Concept** kann zunächst der normale OSM-Tile-Server im Internet verwendet werden, um Entwicklungsaufwand zu sparen. Parallel planen wir jedoch die Einrichtung eines **lokalen Tile-Servers**: Dieser wird die benötigten Kartenausschnitte (z.B. ganz Deutschland oder nur die Region) in einer Datenbank (PostGIS) halten und bei Anfrage Kacheln rendern. Das genannte Tutorial-Setup (Apache + mod_tile + renderd + Mapnik) ist bewährt und Open-Source ². Alternativ gibt es einfachere Möglichkeiten: Man könnte z.B. ein Tool nutzen, das OSM-Kacheln vorgeneriert. Denkbar ist auch die Verwendung von **MBTiles** (ein SQLite-basiertes einzelnes Datenbankfile mit vorgerenderten Kacheln) – dann könnte der Client diese Datei direkt lesen oder ein minimaler Server sie ausliefern.

Darstellung der Einsatzdaten auf der Karte: Die Positionen der Einsatzmittel, die ja regelmäßig per GPS reinkommen, werden als **Layer** auf der Karte angezeigt. Jedes Fahrzeug erhält ein Symbol (Icon), das seinen Typ andeutet (z.B. ein Krankenwagensymbol für RTW). Die Icons können farblich den Status reflektieren (frei = grün, in Einsatz = rot, o.ä.). Der Client abonniert gewissermaßen die Positionsdaten: entweder über Polling (alle X Sekunden Position updaten) oder eventbasiert (z.B. Gateway schreibt neue Position -> DB -> Notify -> Client aktualisiert diesen Marker). Die **Einsatzorte** werden ebenfalls markiert, evtl. mit einem speziellen Pin und Popup, das Details zeigt, wenn man darauf klickt (Adresse, Einsatz-Nr, beteiligte Einheiten etc.). Eine wichtige Funktion ist auch das **Zentrieren/Zoomen**: Wenn ein Disponent einen Einsatz auswählt, sollte die Karte auf dessen Standort springen und die nächsten Einheiten anzeigen.

Falls Routing benötigt wird (z.B. um die *Fahrstrecke* des nächstgelegenen Fahrzeugs zum Einsatzort abzuschätzen), kann man auf OSM-basierte Routing-Engines zurückgreifen. Als Beispiel gibt es **GraphHopper** oder **OSRM** (Open Source Routing Machine), die Routen berechnen können. Diese könnten entweder über den Gateway-Server angebunden werden (externe API oder eigene Instanz) oder – einfacher – man verzichtet im PoC auf automatische Routenberechnung. Der Disponent kann ggf. selbst im Kopf planen oder es manuell auslösen. Langfristig wäre eine Routingintegration aber wünschenswert (ggf. als Modul).

Offline-Karten & Cache: Ein essenzieller Punkt: Wenn das System offline arbeiten soll, müssen Kartendaten verfügbar bleiben. Qt Location Plugin bietet bereits die Möglichkeit, Tiles zwischenspeichern (disk cache) ⁷. Wir können den Client so konfigurieren, dass einmal geladene Kacheln lokal zwischengespeichert werden (im Dateisystem oder RAM) – somit würden zuvor betrachtete Gebiete bei Netzausfall weiterhin angezeigt. Allerdings deckt das nicht neue Gebiete ab. Besser ist die erwähnte **eigene Tile-Server-Lösung** im Intranet. Diese würde im Hintergrund laufend Tiles rendern oder beim ersten Anfordern berechnen und cachen. Eine lokale PostGIS-Datenbank mit OSM-Daten aller relevanten Gebiete stellt sicher, dass auch im Offline-Fall Karten gerendert werden können, da keine externe Anfrage nötig ist. In der Übergangsphase kann man einen hybriden Ansatz nehmen: OSM-Tiles per Internet laden, aber in einer größeren Kachel-Cache-Datenbank speichern (es gibt Tools dafür), um nach und nach das Gebiet abzudecken.

Fazit Kartenmodul: Der Wechsel zu OpenStreetMap bringt viele Vorteile: **Kostenfreiheit, Unabhängigkeit und Flexibilität**. Sämtliche Software dafür ist quelloffen ⁶. Für den Nutzer (Disponent) ändert sich funktional wenig – die Karte lässt sich analog bedienen, eventuell sind die Kartenstile etwas anders als gewohnt, aber anpassbar. Wichtig ist, dass die Performance hoch bleibt: Durch lokale Server und ggf. vektorielle Darstellung (Qt kann auch mit Qt Quick und OpenGL viele Marker performant rendern) wird eine flüssige Karte ermöglicht, die in Echtzeit Positionsänderungen zeigt.

Externe Schnittstellen und Gateway-Server

Wie beschrieben, wird der **Client selbst keine direkten Schnittstellen zur Außenwelt** haben. Stattdessen bündelt ein **Gateway-Server** sämtliche externen Integrationen. Dieses Konzept erhöht die Sicherheit und erlaubt es, extern bezogene Daten zentral aufzubereiten. Im Folgenden wird erläutert, wie die externen Schnittstellen im Detail umgesetzt werden:

- **Wetterdaten-Integration:** Als exemplarische externe Quelle sollen Wetterinformationen (z.B. Temperatur, Wetterlage, Unwetterwarnungen) ins System einfließen. Hierzu wird auf dem Gateway-Server ein **Wetter-Service** eingerichtet. Dies kann z.B. ein Python-Skript oder ein kleiner Daemon sein, der in festen Intervallen (etwa alle 5 Minuten) die API eines Wetterdienstes abfragt. Geeignete APIs wären etwa *OpenWeatherMap* (erfordert API-Key, bietet aktuelle Werte und Prognosen) oder nationale Dienste (Deutscher Wetterdienst hat Open-Data-Schnittstellen). Das Gateway-Skript holt z.B. die aktuelle Temperatur, Regenwahrscheinlichkeit usw. für definierte Orte (z.B. Leitstellenbezirk oder wichtige Punkte) und schreibt diese Werte in die **Wetter-Tabelle** der Leitstellen-DB. Ein Eintrag könnte Felder haben wie Timestamp, Ort, Temperatur, Wetterzustand ("bewölkt") etc. Der Client kann diese Daten anzeigen, z.B. im Hauptfenster einen kleinen Wetter-Info-Bereich. Falls die Internetverbindung wegfällt, liefert das Skript keine neuen Daten – aber der Client behält den letzten Stand, sodass zumindest veraltete (aber kürzlich gültige) Info vorhanden ist.
- **Weitere mögliche Schnittstellen:** Der Gateway-Server kann modular erweitert werden. Denkbar sind:
 - **Geocoding** (Adressauflösung): Falls der Disponent eine Adresse eingibt, könnte ein Geocoding-Dienst (z.B. Nominatim von OSM) die Koordinaten liefern. Solche Abfragen könnte man auch via Gateway leiten (eingetippte Adresse -> Gateway -> Nominatim API -> Koordinate zurück -> Speicherung in DB).
 - **Reverse-Geocoding** (Rückwärtssuche Adresse zu Koordinate): Wenn ein Handy-Notruf direkt Koordinaten liefert (AML, Advanced Mobile Location), könnte man eine Adressbeschreibung via Gateway holen.
 - **Alarmierungssysteme:** Ansteuerung von Meldeempfängern oder SMS/Push an Einsatzkräfte könnte über einen externen Dienst laufen – würde dann ebenfalls durch ein Gateway-Modul gekapselt und über die DB oder definierte Queues ausgelöst.
 - **Behördenetze:** Schnittstellen zu Polizei-/Feuerwehr-Leitstellen anderer Kreise oder zentrale Notruf-Weiterleitungen (wie in NexSIS geplant) könnten über standardisierte Protokolle (z.B. EDXL, CAP) laufen. Diese würde man auch als separate Services implementieren, die an das System andocken, ohne den Client-Code zu verändern (der Client sieht nur, dass z.B. ein Einsatz hereinkommt, weil das Gateway ihn in die DB geschrieben hat).
 - **Kommunikation und Protokoll:** Der Gateway-Server kann auf zwei Arten mit der DB interagieren: direkt auf DB-Ebene (als DB-Client mit Schreibrechten) oder über eine definierte API des DB-Servers. Für unser Setup ist es unkompliziert, dem Gateway direkten DB-Zugriff zu geben (er läuft ja auch im internen Netz). Alternativ könnte man einen kleinen Webservice auf dem Datenbankserver bereitstellen (z.B. REST-API via Python/Flask oder Node.js), über den der Gateway seine Daten einstellt. Das ist jedoch ein unnötiger Overhead, solange die Sicherheit intern gewährleistet ist. Daher: das Gateway-Skript nutzt vermutlich einfach einen PostgreSQL-Client (z.B. psycopg2 in Python), um seine INSERT/UPDATES zu machen.
 - **Security des Gateways:** Da der Gateway-Server nach außen kommuniziert, muss er besonders gehärtet werden. Nur erlaubte Ziele (konfigurierte API-Endpoints) sollen kontaktierbar sein, idealerweise über HTTPS mit Zertifikatsprüfung. Die Credentials (API-Schlüssel, Passwörter) werden sicher auf dem Gateway gespeichert (z.B. in Ansible Vault oder als Umgebungsvariablen, nicht im Klartext im Code). Der Gateway-Server selbst sollte keine eingehenden Ports offen haben außer evtl. SSH für Wartung – alle Kommunikation geht von ihm aus und er schreibt in die

DB. Man kann an der Firewall erzwingen, dass nur der Gateway-Host aus dem Segment ins Internet darf, die Clients und DB-Server hingegen nicht.

Durch diese Architektur kann der **Leitstellen-Client komplett API-frei bleiben** – für ihn ist es so, als stünden alle Daten lokal in der Datenbank bereit. Das vereinfacht die Client-Entwicklung und erhöht die Robustheit. Gleichzeitig können aber alle notwendigen externen Informationen integriert werden, indem man einfach das Gateway erweitert. Die Trennung von Verantwortlichkeiten (Client für Darstellung/Interaktion, Server-DB für Daten, Gateway für externe Kommunikation) macht das System auch **flexibel erweiterbar**.

Sicherheit und Ausfallsicherheit

Im Leitstellenbetrieb hat **Betriebssicherheit** oberste Priorität – das System muss zuverlässig laufen, und sensible Daten müssen geschützt sein. Entsprechend sind folgende Sicherheits- und Failover-Aspekte im Konzept verankert:

- **Netzwerk und Zugriffsschutz:** Die Datenbank sowie der Gateway-Server laufen in einem gesicherten internen Netzwerk/VLAN. Nur autorisierte Clients (Leitstellen-Arbeitsplätze) können auf die DB zugreifen – z.B. durch Firewall-Regeln oder VPN-Tunnel bei verteilten Standorten. Auf DB-Ebene werden Rollen eingerichtet, so dass der Client nur die benötigten Rechte hat (idealerweise Prozeduren aufruft, anstatt freie SQLs, aber im PoC evtl. einfacher). Der Gateway hat ebenfalls definierte DB-Rechte für die Tabellen, die er beschreibt.
- **Authentifizierung und Protokollierung:** Jeder Benutzer (Disponent) meldet sich am Client mit Benutzername/Passwort an. Aktionen im System (Einsatz anlegen, Status ändern etc.) werden mit Benutzer-ID und Zeit protokolliert (Audit Trail), um Missbrauch oder Fehler nachverfolgen zu können. Ansible kann genutzt werden, um initial Benutzerkonten in der DB anzulegen. Passwörter werden nur gehasht gespeichert (z.B. bcrypt).
- **Datenverschlüsselung:** Die Kommunikation zwischen Client und DB-Server kann mittels TLS verschlüsselt werden (PostgreSQL unterstützt SSL-Verbindungen). Damit sind auch in einem internen Netz die Daten vor Abhören geschützt, falls z.B. jemand ein Gerät ins Netz hängt. Zudem sollte der Gateway externe Verbindungen immer verschlüsselt (HTTPS) halten.
- **Physische Ausfallsicherheit:** Damit das System auch bei Hardwareproblemen weiterläuft, sind Maßnahmen wie RAID für Server-Disks, Notstromversorgung (USV) und regelmäßige Backups vorgesehen. In einem erweiterten Szenario könnten zwei Server im Redundanzverbund laufen (z.B. ein zweiter DB-Server, wie oben erwähnt, für Failover via Patroni oder ähnliche Tools). Für den PoC genügt eine einzelne VM, aber im Konzept ist skizziert, wie man später auf hochverfügbar (HA) gehen kann.
- **Daten-Backups:** Automatisierte Backups der Datenbank (z.B. jede Nacht oder inkrementell häufiger) stellen sicher, dass im Worst Case kein großer Datenverlust eintritt. Backups werden idealerweise auf ein zweites System oder externes Laufwerk gespeichert. Ansible kann Backup-Jobs (Cron oder systemd timer) einrichten.
- **Notfallmodus und Fallbacks:** Sollte dennoch die Datenbank ausfallen oder unzugänglich sein, müsste ein **Notfallbetrieb** definiert werden. Z.B. könnten die Disponenten auf Papierlogging umstellen, bis das System wieder hochfährt. Da dies aber sehr unwünschenswert ist, tut man alles, um die DB erreichbar zu halten (Monitoring einrichten, Alarme, etc.).
- **Updates und Patches:** Durch den Einsatz gängiger Open-Source-Software ist es wichtig, Updates einzuspielen (z.B. Security-Patches für PostgreSQL oder Qt-Bibliotheken). Ansible erleichtert dies enorm – man kann Playbooks schreiben, die Pakete aktualisieren oder neue Client-Builds ausrollen, mit minimaler Downtime.
- **Keine Single Points of Failure beim Gateway:** Der Gateway-Server Ausfall hätte zwar nicht unmittelbaren Einfluss auf die laufende Einsatzabwicklung (der Client und die DB würden weiter

funktionieren), aber externe Infos fehlen dann. Trotzdem sollte der Gateway ebenfalls redundant oder schnell wiederherstellbar sein. Da er zustandslos ist (sein Zustand ist ja in der DB hinterlegt), kann man ihn notfalls neu starten oder parallel einen zweiten kalt standby halten, der einspringt, falls der erste ausfällt.

Zusammengefasst: Das Systemdesign stellt sicher, dass *lokal weitergearbeitet* werden kann, selbst wenn z.B. das Internet wegbricht. Alle operativen Daten liegen in der eigenen Datenbank vor – das umfasst alle Einsatzinformationen, Ressourcenstatus und im Idealfall auch alle notwendigen Hintergrunddaten (Karten, Adressdatenbank, etc.). Externe Dienste werden nur hinzugeschaltet, aber die Leitstelle ist **nicht von ihnen abhängig**. Damit wird dem Sicherheitsgrundsatz entsprochen, dass die Leitstelle auch im Krisenfall (wenn etwa ein größerer Stromausfall oder Cyberangriff außerhalb vorliegt) autonom bleibt und Notrufe disponieren kann.

Deployment-Ablauf mit Ansible

Für die Installation und Konfiguration des Gesamtsystems kommen **Ansible Playbooks** zum Einsatz, die in Rollen aufgeteilt sind. Dies gewährleistet eine einheitliche, reproduzierbare Einrichtung auf allen Servern und Clients. Geplant sind u.a. folgende Rollen:

- **Datenbank-Server Rolle:** Diese Ansible-Rolle installiert PostgreSQL (auf Rocky Linux) und richtet die Leitstellen-Datenbank ein. Schritte: PostgreSQL Server Paket installieren, Dienst aktivieren, Admin-Benutzer setzen. Dann mittels Ansible-Module oder SQL-Skripten das Datenbankschema anlegen (Tabellen, Indizes, evtl. PostGIS Extension installieren). Des Weiteren erstellt die Rolle die notwendigen DB-User/Rollen für den Leitstellen-Client und Gateway mit passendem Rechte-Setup. Konfigurationsdateien wie `postgresql.conf` und `pg_hba.conf` werden angepasst (z.B. Tunings für Performance, Zulassen der Client-Netzwerkadressen). Am Ende läuft der DB-Server mit dem vordefinierten Schema.
- **Client-Installationsrolle:** Diese Rolle kümmert sich um die Arbeitsplätze (kann gruppiert nach OS-Familie). Beispielsweise für RHEL-basierte Leitstellen-PCs: Installieren benötigter Bibliotheken (Qt Runtime, falls dynamisch gelinkt, oder sonst grundlegende Pakete wie xcb, OpenGL libs usw.), Deployment der eigentlichen Client-Software (entweder Kopieren eines AppImage oder Installation eines RPM/DEB). Ansible kann z.B. die neueste Version vom Build-Server holen. Danach kann die Rolle noch Desktop-Einträge erstellen (Verknüpfung im Startmenü) und ggf. Autostart-Konfiguration, falls der Client beim Systemstart hochfahren soll. Für Debian/Ubuntu-Systeme macht die Rolle analog, nur mit apt statt yum. Durch Variablen in Ansible lässt sich differenzieren, sodass derselbe Play je nach Hosttyp die richtigen Schritte ausführt.
- **Gateway-Server Rolle:** Diese installiert auf dem vorgesehenen Gateway-VM die benötigten Tools (z.B. Python3, pip Packages für API-Abfragen wie `requests` für HTTP, evtl. `psycopg2` für DB-Zugriff). Sie legt das Verzeichnis für Gateway-Skripte an und kopiert die Skripte (Wetterdienst etc.) dorthin. Außerdem wird ein Systemdienst eingerichtet (etwa ein systemd Service oder Timer), der diese Skripte regelmäßig ausführt. Z.B. könnte man einen systemd Timer definieren, der alle 5 Minuten das Wetter-Skript startet. Ansible schreibt die entsprechende Unit-Datei und aktiviert sie. Auch API Keys oder Konfigurationswerte können via Ansible Vault sicher hinterlegt und dann beim Deployment in Config-Files des Gateways entpackt werden.
- **(optional) Tile-Server Rolle:** Falls wir einen eigenen OSM-Server aufsetzen, gäbe es hierfür eine dedizierte Rolle. Diese würde PostGIS installieren, dann das gewünschte OSM-Datenauszug laden (z.B. via `osm2pgsql` importieren), und einen Renderdienst installieren (Apache + `mod_tile` + `renderd`, laut OSM Doku). Das ist recht komplex, aber es existieren Community-Rollen/Skripte, die man nutzen oder anpassen kann. Für den PoC wird diese Rolle evtl. noch nicht voll umgesetzt, aber im Konzept vorgesehen, um später offline Karten anbieten zu können.

Die Deployment-Pipeline könnte folgendermaßen aussehen: Ein **Master-Playbook** orchestriert die Rollen in richtiger Reihenfolge. Zuerst wird der Datenbank-Server konfiguriert, dann der Tile-Server (falls vorhanden), dann der Gateway, zuletzt die Clients. Ansible kann parallel auf mehrere Client-Hosts ausgerollt werden, was bei z.B. 5 Disponenten-Arbeitsplätzen effizient ist.

Auch Updates laufen über Ansible: Möchte man eine neue Client-Version deployen, ersetzt man im Repository die entsprechende Binary und führt die Client-Rolle erneut aus (diese erkennt Version und aktualisiert). Gleiches für Schema-Änderungen: DB-Änderungen können als migrations-Skripte in der DB-Rolle gepflegt werden.

Versionsverwaltung und Idempotenz sind hier wichtig – Ansible gewährleistet, dass bei wiederholtem Lauf der Playbooks der Zustand konsistent bleibt (z.B. Tabellen nicht jedes Mal neu angelegt werden, sondern nur wenn nicht vorhanden oder wenn Schema-Änderungen vorliegen).

Zudem ermöglicht dieses Setup auch das **Testen in isolierten Umgebungen**: Man könnte in VMware eine Staging-Umgebung (getrenntes Netzwerk) hochziehen, ansible darüber laufen lassen und das Setup testen, bevor es in die echte Leitstelle geht.

Erweiterbarkeit und Zukunftsperspektive

Das vorgeschlagene Konzept legt den Grundstein für ein flexibles Leitstellensystem, das mit zukünftigen Anforderungen mitwachsen kann. Einige Perspektiven:

- **Modulare Erweiterungen:** Dank der modularen Architektur (Trennung von Client, DB, Gateways) lassen sich neue Module vergleichsweise leicht hinzufügen. Beispielsweise könnte ein **Alarmierungsmodul** integriert werden, das bei Alarm bestimmte SMS oder Pager-Signale auslöst – hierfür würde man ein Gateway-Skript schreiben, das mit einem Alarmserver (oder direkt mit Funk-Pager-Schnittstelle) kommuniziert. Der Client bekäme ggf. einen Button "Alarmierung auslösen", der intern in der DB einen Status setzt, worauf das Gateway reagiert. Ähnlich könnten **Schnittstellen zu Nachbarleitstellen** realisiert werden (z.B. gemeinsame Einsätze via Datenaustausch). Die Architektur isoliert solche Integrationen, sodass der Kern stabil bleibt.
- **Skalierung und Performance-Tuning:** Sollte das System in einem größeren Rahmen eingesetzt werden (etwa mehrere Leitstellen oder sehr viele gleichzeitige Nutzer), kann es skaliert werden. Die in Frankreich entwickelte NexSIS-Plattform z.B. zielt auf bis zu 250k simultane Nutzer und setzt auf eine webbasierte Cloud-Architektur ⁸. Unser System könnte durch *Lastverteilung* ebenfalls skaliert werden: Mehrere Instanzen des Clients sind ohnehin möglich; für die DB könnte man Sharding oder Load Balancer einsetzen, falls nötig (unwahrscheinlich für einen Kreis). Die Gateway-Module kann man parallelisieren (mehrere Threads oder Dienste, die verschiedene Aufgaben übernehmen). Durch Monitoring der Systemauslastung können Engpässe identifiziert und adressiert werden (z.B. Query-Optimierung, Caching etc.).
- **Benutzeroberfläche und UX:** Zukünftig kann die GUI noch weiter verfeinert werden: Unterstützung von Mehrbildschirm-Arbeitsplätzen (Karte auf separatem Monitor), konfigurierbare Dashboard-Elemente, Integration von Video (z.B. Übertragung von Notruf-Video falls 112-Apps das senden – NexSIS plant so etwas). Solche Funktionen lassen sich in einem Qt-Client nachrüsten (Qt hat Multimedia-Module). Wichtig ist, von Beginn an eine saubere Struktur zu haben, um neue Views oder Fenster leicht anzuhängen.
- **Cross-Plattform und mobile Clients:** Obwohl der Fokus auf Linux liegt, ließe sich perspektivisch auch ein **schaffen. Etwa eine Tablet-App für den LNA (Leitender Notarzt) oder für Einsatzleiter vor Ort, die einen Überblick erlaubt. Da wir offene Standards nutzen, könnte so**

ein mobiler Client entweder direkt auf die gleiche DB (über VPN) zugreifen oder über einen Webservice. Qt ist in der Lage, auch Android/iOS-Apps zu erstellen, falls das gewollt ist – ein späterer Schritt könnte also sein, denselben Qt-Code für Tablets zu kompilieren. Alternativ, mit einer Web-UI-Variante wäre die mobile Unterstützung noch einfacher (Browser auf Tablet).

- **Open-Source-Community:** Wenn das Projekt erfolgreich ist, besteht die Möglichkeit, es als Open-Source-Leitstellensoftware anderen Kreisen zur Verfügung zu stellen. Bereits jetzt gibt es Projekte wie Resgrid (US) oder Ansätze im europäischen Raum. Unser Ansatz nutzt konsequent Open-Source, was potentiell Interessierte anzieht. Eine Community könnte bei Weiterentwicklung helfen, Module beitragen (z.B. ein Einsatz-Nachbearbeitungsmodul, Statistik/Reporting, eCall-Integration etc.). Die Offenheit des Systems ist somit ein strategischer Vorteil.
- **Compliance und Standards:** Zukünftig könnten gesetzliche Anforderungen entstehen, z.B. Anbindung an ein zentrales Notrufrouting oder Einhaltung bestimmter IT-Sicherheitsrichtlinien. Durch die Kontrolle über alle Komponenten können wir das System gezielt anpassen. Der Einsatz offener Standards (wie OGC-Standards für Geodaten, CAP für Alarmmeldungen etc.) erleichtert die Interoperabilität. Sollte z.B. NG112 (Next Generation 112) Standardfunktionen verlangen (wie Videoanrufe, Chats), kann man diese als separate Module implementieren, die mit unserem System interagieren.

Zusammenfassung: Der vorgestellte Plan zeigt, dass ein **performanter Linux-Client für Leitstellen** durchaus realisierbar ist – zumal die bestehende proprietäre Software ebenfalls auf Linux (Rocky) läuft und somit keine prinzipiellen Hürden bestehen. Durch geschickte Wahl von Technologien (Qt/C++ für schnellen Client, PostgreSQL für robuste Datenbasis, OpenStreetMap für freie Kartennutzung) und eine klare Schichtenarchitektur erreichen wir ein System, das **schnell, sicher und erweiterbar** ist.

Die Umsetzung als Ansible-getriebenes Deployment stellt sicher, dass das System konsistent aufgesetzt werden kann und im Betrieb leicht zu warten ist. Die Fokussierung auf lokale Datenhaltung gewährleistet Unabhängigkeit von der Außenwelt, was gerade für Notfallsysteme essentiell ist. Gleichzeitig gehen wir keine Kompromisse bei modernen Features ein – Integration von Echtzeit-Karten, externen Daten wie Wetter, und Möglichkeiten zur zukünftigen Erweiterung sind gegeben.

Der nächste Schritt ist nun, diesen Plan in einen Prototypen zu überführen: Die Grundfunktionalitäten (Einsatz erstellen, Status ändern, Karte mit OSM anzeigen, einfache Wetteranzeige) sollten implementiert und auf der vorgesehenen Infrastruktur ausgerollt werden. Damit kann die Leistungsfähigkeit des Ansatzes demonstriert werden. Gelingt der PoC, steht einer schrittweisen Ablösung der bisherigen Software durch dieses eigene Leitstellensystem nichts im Wege. Das Endergebnis wäre ein **maßgeschneidertes, kostengünstiges Einsatzleitsystem**, das den Anforderungen von 2025 und darüber hinaus gerecht wird.

Quellen: Diese Konzeption stützt sich auf aktuelle Best Practices und Erfahrungen ähnlicher Projekte. Zum Beispiel setzt das französische NexSIS-Leitstellensystem gezielt auf Open-Source-Technologien (insb. beim Kartensystem) und eine webbasierte Architektur ¹. Für die technische Umsetzung bieten etablierte Open-Source-Tools die Grundlage: PostgreSQL/PostGIS als schnelle, räumlich erweiterbare Datenbank ² (die in Benchmarks gegenüber MySQL/MariaDB massive Geschwindigkeitsvorteile zeigte ⁵), sowie Qt als plattformübergreifendes Framework mit nativer OSM-Karten-Unterstützung ³. All diese Komponenten sind frei verfügbar ⁶ und haben sich in vielen Anwendungen bewährt. Dieses Konzept vereint sie zu einer auf die Leitstelle zugeschnittenen Lösung.

¹ ⁸ Einsatzleitsystem Archive – Sprechauufforderung

<https://www.sprechauufforderung.de/tag/einsatzleitsystem/>

2 6 Installing an OpenStreetMap Tile Server on Ubuntu | OpenStreetMap Carto Tutorials

<https://ircama.github.io/osm-carto-tutorials/tile-server-ubuntu/>

3 7 Qt Location Open Street Map Plugin | Qt Location | Qt 6.9.1

<https://doc.qt.io/qt-6/location-plugin-osm.html>

4 Database with PostgreSQL | Oracle

<https://www.oracle.com/cloud/postgresql/>

5 Fastest Open-Source Databases | Data System Reviews

<https://datasystemreviews.com/fastest-open-source-databases.html>